

## 0.17.3. La fonction `deplace_x()` — avancer sur une ligne

`#include <deplace_x>` : `deplace_x(x, y, tuile, pas)` fait avancer la tuile de « pas » cases vers la droite (ou la gauche si pas est négatif), un pas tous les quarts de seconde.

### 1. Ce qu'il faut comprendre

**Ce qu'elle fait** : `deplace_x(x, y, tuile, pas)` pose la tuile en (x, y), puis la fait **avancer** de `pas` cases : vers la droite si `pas` est positif, vers la gauche s'il est négatif. Un pas tous les **250 ms**. Elle **bloque** : le programme attend la fin du trajet.

**La ligne à écrire** : `#include <deplace_x>`.

**Ses arguments** : la colonne et la ligne de départ, la tuile, le nombre de pas. Seule sur sa ligne, avec une variable en premier, elle **range l'arrivée** dans cette variable.

**Ce qu'elle coûte** : environ **220 octets** pour elle-même, et la police entière (**704 octets**) : la tuile arrive par un argument, le compilateur ne sait pas laquelle ce sera.

**Essaie** : `deplace_x(x, 8, ALPHABET[0], -5);` après le premier : le A revient.

#### On retrouve `deplace_x` dans 24 leçons

[0.18](#), [0.18.1](#), [0.18.2](#), [0.19](#), [0.19.1](#), [0.19.2](#), [0.20](#), [0.20.1](#), [0.20.2](#), [0.23](#), [0.23.1](#), [0.23.2](#), [0.24](#), [0.24.1](#), [0.24.2](#), [0.25](#), [0.25.1](#), [0.25.2](#), [0.55](#), [0.55.1](#), [0.55.2](#), [0.56](#), [0.56.1](#), [0.56.2](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1068 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                |       |                              |                                                     |
|----------------|-------|------------------------------|-----------------------------------------------------|
| démarrage      | 60 o  | ← la console, à l'allumage   | – la mise en route de la console                    |
| deplace_x()    | 158 o | ← main()                     | – une fonction fournie par le langage               |
| main()         | 28 o  | ← démarrage                  | – ta fonction                                       |
| VBlank         | 8 o   | ← la console, à chaque image | – compte les images, 60 fois par seconde            |
| AttendreImage  | 28 o  | ← image()                    | – attend l'image suivante                           |
| AttendreVBlank | 16 o  | ← démarrage                  | – attend que l'écran soit libre                     |
| AttendreAcces  | 12 o  | ← EffacerCases               | – attend de pouvoir écrire une case                 |
| AdresseCase    | 10 o  | ← poser(), effacer()         | – trouve l'adresse d'une case de l'écran            |
| EffacerCases   | 12 o  | ← effacer()                  | – efface des cases                                  |
| CopierTuiles   | 18 o  | ← démarrage                  | – copie les lettres et les dessins en mémoire vidéo |
| EffacerCarte   | 14 o  | ← démarrage                  | – vide les deux cartes de l'écran                   |
| Tuiles         | 704 o | ← CopierTuiles               | – la police entière                                 |

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()      × 2    26 o sur place, dans deplace_x() + AdresseCase (10 o)
effacer()    × 1    15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      × 2     6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() × 1    19 o sur place, dans main()
```

### 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <deplace_x> : avancer sur une ligne ----
// Le A part de la colonne 2 et avance de 10 cases : il finit en colonne 12.

#include <deplace_x>    // deplace_x() : avancer sur la ligne
#include <ALPHABET>
```

### 2. UNE VARIABLE — X

```
uint8_t x = 2;
```

### 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace_x(x, 8, ALPHABET[0], 10);    // 10 pas vers la droite ; x reçoit 12

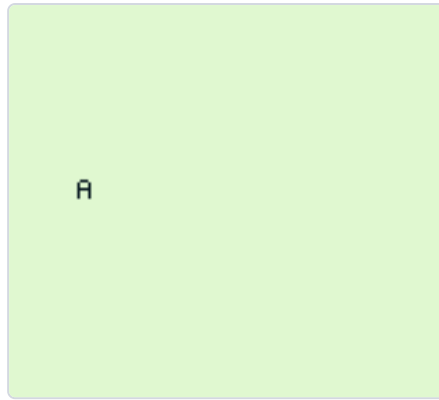
    while (true) {
        image();
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



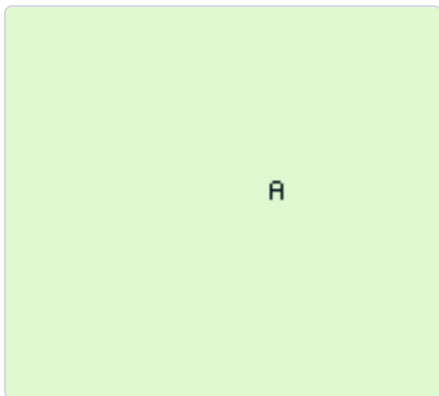
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu'on doit voir

Un A qui avance de dix cases vers la droite, un pas par quart de seconde.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (12, 8) (12,8)

✓ x a reçu l'arrivée : 12

Ensuite : **0.18. Avancer avec `deplace_x`** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++